

Functions

Introduction

A **function** is a reusable block of code that performs a specific task.

Instead of writing the same code multiple times, we place it inside a function and call it whenever needed.

Functions improve code reusability, readability, and maintainability.

Advantages of Functions

- Code Reusability
- Better Code Organization
- Reduces Code Duplication
- Easier Debugging
- Easier Maintenance
- Improves Readability
- Makes Programs Modular

Real-Time Example

Imagine an e-commerce website.

Whenever a customer places an order, the system calculates GST.

Instead of writing the GST calculation code everywhere, we create a function.

Python

```
def calculate_gst(amount):  
    gst = amount * 0.18  
    print("GST:", gst)
```

```
calculate_gst(1000)  
calculate_gst(5000)
```

Types of Functions

Functions in Python are mainly classified into four types.

1. Built-in Functions

These functions are already provided by Python.

We can use them directly without creating our own functions.

Common Built-in Functions

Function	Purpose	Example
<code>print()</code>	Displays output	<code>print("Hello")</code>
<code>len()</code>	Returns the length of an object	<code>len("Python")</code>
<code>min()</code>	Returns the smallest value	<code>min(5, 2, 8)</code>
<code>max()</code>	Returns the largest value	<code>max(5, 2, 8)</code>
<code>abs()</code>	Returns the absolute value	<code>abs(-15)</code>
<code>sum()</code>	Returns the sum of elements	<code>sum([1, 2, 3])</code>
<code>sorted()</code>	Returns a sorted list	<code>sorted([3, 1, 2])</code>
<code>type()</code>	Returns the data type	<code>type(10)</code>
<code>round()</code>	Rounds a number	<code>round(4.76)</code>

Example

Python

```
print(len("Codegnan"))
print(max(5, 12, 9))
print(abs(-100))
print(sorted([8, 3, 6, 1]))
```

2. User Defined Functions

Functions created by programmers are called **User Defined Functions**.

Syntax

```
Python
def function_name():
    statements
```

Example

```
Python
def welcome():
    print("Welcome to Codegnan")
welcome()

# Output:
# Welcome to Codegnan
```

3. Anonymous (Lambda) Functions

Functions without a name are called **Anonymous Functions** or **Lambda Functions**.

They are created using the `lambda` keyword.

Syntax

```
Python
lambda parameters: expression
```

Example

```
Python
square = lambda x: x * x
print(square(5))

# Output:
# 25
```

4. Recursive Functions

A function that calls itself is called a **Recursive Function**.

Recursive functions are commonly used to solve problems such as factorial, Fibonacci series, and tree traversal.

Syntax

Python

```
def function_name(parameters):  
    if base_condition:  
        return value  
    return function_name(modified_parameters)
```

Example

Python

```
def factorial(n):  
    if n == 1:  
        return 1  
    return n * factorial(n - 1)  
  
print(factorial(5))
```

Function Syntax

General Syntax

Python

```
def function_name(parameters):  
    """  
    Docstring explaining the purpose of the function.  
    """  
    statements  
    return value  
function_name(arguments)
```

Components of a Function

- **def** → Keyword used to define a function.
- **function_name** → Name of the function.
- **parameters** → Input values accepted by the function.
- **Docstring** → Explains the purpose of the function.
- **Statements** → Body of the function.
- **return** → Sends a value back to the caller.
- **Function Call** → Executes the function.

Example

Python

```
def square(number):  
    """  
    Returns the square of a number.  
    """  
    result = number * number  
    return result  
print(square(5))
```

Return Statement

Definition

The **return** statement sends a value back to the function caller. Once the **return** statement executes, the function immediately stops.

Syntax

Python

```
return value
```

Example

Python

```
def get_price():  
    return 999  
price = get_price()  
print(price)
```

Multiple Return Values

Syntax

Python

```
return value1, value2, value3
```

Example

Python

```
def student():  
    return "Teja", 95  
  
name, marks = student()  
print(name)  
print(marks)
```

Function Arguments

Python supports different types of arguments.

- Positional Arguments
- Default Arguments
- Keyword Arguments
- Variable Length Arguments (**args*)
- Keyword Variable Length Arguments (***kwargs*)

Positional Arguments

Definition

Arguments are assigned according to their position.

The order of arguments must be maintained.

Syntax

Python

```
function(arg1, arg2)
```

Example

Python

```
def login(username, password):  
  
    print("Username:", username)  
    print("Password:", password)  
  
login("admin", "admin123")  
  
# Output:  
# Username: admin  
# Password: admin123
```

Wrong Example

Python

```
login("admin123", "admin")
```

Values are assigned based on their position.

Default Arguments

Definition

A default value is assigned to a parameter.

If no value is passed during the function call, the default value is used.

Syntax

Python

```
def function(parameter=value):
```

Example

Python

```
def delivery_charge(charge=40):  
  
    print("Delivery Charge:", charge)  
  
delivery_charge()  
delivery_charge(80)  
  
# Output:  
# Delivery Charge: 40  
# Delivery Charge: 80
```

Keyword Arguments

Definition

Arguments are passed using parameter names.

The order of arguments does not matter.

Syntax

Python

```
function(parameter=value)
```

Example

Python

```
def employee(name, salary):  
  
    print(name)  
    print(salary)  
  
employee(salary=50000, name="Rahul")  
  
# Output:  
# Rahul  
# 50000
```


Variable Length Arguments (*args)

Definition

Used when the number of positional arguments is unknown.

Python stores all arguments inside a tuple.

Syntax

Python

```
def function(*args):
```

Example

Python

```
def total_bill(*prices):  
  
    print(prices)  
  
total_bill(500, 1000, 1500)  
  
# Output:  
# (500, 1000, 1500)
```

Real-Time Example

Python

```
def total_bill(*prices):  
  
    print("Total:", sum(prices))  
  
total_bill(500, 1000, 1500)  
  
# Output:  
# Total: 3000
```

Keyword Variable Length Arguments (**kwargs)

Definition

Used when the number of keyword arguments is unknown.

Python stores all keyword arguments inside a dictionary.

Syntax

Python

```
def function(**kwargs):
```

Example

Python

```
def profile(**details):
```

```
    print(details)
```

```
profile(
    name="Teja",
    age=23,
    city="Hyderabad"
)
```

```
# Output:
```

```
# {'name': 'Teja', 'age': 23, 'city': 'Hyderabad'}
```

Real-Time Example

Python

```
def product(**details):
```

```
    for key, value in details.items():
```

```
        print(key, ":", value)
```

```
product(  
    name="Laptop",  
    brand="HP",  
    price=65000  
)
```

```
# Output:  
# name : Laptop  
# brand : HP  
# price : 65000
```

Difference Between *args and **kwargs

Feature	*args	**kwargs
Full Form	Arbitrary Arguments	Arbitrary Keyword Arguments
Symbol	*	**
Receives	Positional Arguments	Keyword Arguments
Stores Data In	Tuple	Dictionary
Example	(10, 20, 30)	{'name': 'Teja'}